

Team 7 - E-Waste Battery Extraction

CV-21: Inference Performance Profiling & Optimisation

Full Technical Implementation Report

ROS 2 Jazzy, Gazebo, ONNX Runtime, Profiling and Performance
Engineering

Ubuntu 24.04 on WSL2

Senior Capstone - Sprint 2 Technical Contribution

September 2026

Contents

1	Overview	2
2	System Context	2
3	Benchmark Environment	2
4	Part A: Instrumentation	3
5	Part A: Baseline Collection	4
6	Part A: Startup Outlier	4
7	Part B: Experimental Strategy	5
8	Warm-up Experiment	5
9	Debug Image I/O Experiment	5
10	Threading Experiment	6
11	Execution Provider Experiment	6
12	Preprocessing and Tensor Layout Experiment	7
13	Final Selected Configuration	8
14	Final Controlled Validation	8
15	Relationship Between the Part A and Part B Baselines	10
16	Engineering Decisions	11
17	Evidence and Reproducibility	11
18	Final Technical Outcome	12
19	Conclusion	12

1 Overview

CV-21 is the **Inference Performance Profiling & Optimisation** task for the Team 7 computer-vision pipeline. Its purpose is to profile preprocessing, ONNX inference, post-processing, publishing, and end-to-end performance, then investigate practical optimisation strategies including provider settings, threading, warm-up, and unnecessary image work.

This report combines two technical phases:

- **Part A – Instrumentation and Baseline Benchmarking:** add detailed timing instrumentation, establish a frozen baseline, and identify the dominant bottlenecks.
- **Part B – Optimisation and Validation:** test candidate optimisations one factor at a time, retain only evidence-supported changes, and validate the selected configuration in the live Gazebo/ROS 2 pipeline.

The combined work therefore follows a complete performance-engineering cycle:

Instrument → Measure → Identify Bottlenecks → Optimise → Validate

2 System Context

The perception workspace is located at:

```
/mnt/e/team7_project_bundle/team7_perception_ws/
```

The main files relevant to CV-21 were:

```
inference.py
onnx_phone_detector_node.py
geometry.py
```

The pipeline receives the Gazebo overhead-camera image, converts the ROS image to OpenCV format, performs YOLO11n-seg ONNX inference, reconstructs masks, computes bounding-box / minimum-area-rectangle information, builds the ROS detection message, and publishes the result.

The overhead camera is 1280x960. The model input is 640x640 RGB float32 in NCHW layout.

3 Benchmark Environment

The recorded runtime environment was:

- Python 3.12.3;
- ONNX Runtime 1.29.0;

- available providers: AzureExecutionProvider and CPUExecutionProvider;
- no CUDAExecutionProvider;
- Intel Core i5-9300H @ 2.40 GHz;
- 4 physical CPU cores / 8 logical CPUs;
- ONNX Runtime graph optimisation: `ORT_ENABLE_ALL`;
- default execution mode: `ORT_SEQUENTIAL`.

A consistent terminal convention was used:

- Terminal 1: Gazebo / simulation;
- Terminal 2: ONNX detector node;
- Terminal 3: benchmark control, analysis, statistics, and code changes.

4 Part A: Instrumentation

Before modifying the pipeline, baseline copies of the key source files were preserved. The inference function and ROS callback were then instrumented with `time.perf_counter()`.

The inference function recorded:

```
preprocess_ms
inference_ms
postprocess_ms
```

The ROS node recorded:

```
bridge_ms
raw_save_ms
preprocess_ms
inference_ms
postprocess_ms
annotation_ms
annotated_save_ms
message_build_ms
metrics_csv_ms
publish_ms
self_check_ms
callback_total_ms
processing_fps
```

This design separated the actual model stages from surrounding debug and ROS work. It also isolated the existing CV-17 self-check so that its first-run cost would not be mistaken for normal inference latency.

During implementation, a tab/space indentation error was found and corrected. Both modified Python files then passed `python3 -m py_compile`, and the ROS workspace rebuilt successfully.

5 Part A: Baseline Collection

The instrumented detector produced one timing row per processed frame. A total of 221 measured frames were collected. The first 20 were retained in the raw file but excluded from the steady-state statistics, leaving 201 steady-state frames.

The archived Part A steady-state results were:

Metric	Part A baseline
Steady-state frames	201
Mean callback latency	545.371 ms
Median callback latency	518.476 ms
P95 callback latency	825.621 ms
Mean processing FPS	1.943

The mean stage latencies showed that ONNX inference was the dominant cost:

Stage	Mean latency	Share of callback
ONNX inference	374.840 ms	68.73%
Annotation drawing	67.998 ms	12.47%
Raw PNG saving	34.424 ms	6.31%
Annotated PNG saving	31.026 ms	5.69%
Post-processing	19.352 ms	3.55%
Preprocessing	12.474 ms	2.29%
ROS image conversion	4.137 ms	0.76%
Message construction	0.361 ms	0.07%
Metrics CSV writing	0.295 ms	0.05%
ROS publishing	0.286 ms	0.05%

The profiling therefore identified two clear optimisation targets:

1. ONNX Runtime execution behaviour;
2. non-essential per-frame annotation and image saving.

6 Part A: Startup Outlier

The maximum callback across all recorded frames was 2571.722 ms. The associated CV-17 self-check took approximately 2079.235 ms. In steady state, the self-check mean was approximately 0.002 ms.

This demonstrated why startup frames needed to be separated from steady-state performance and motivated the Part B warm-up experiment.

7 Part B: Experimental Strategy

Part B was performed in a separate evidence directory and used source snapshots before significant modifications. Candidate optimisations were tested individually before being combined.

The main investigations were:

1. warm-up behaviour;
2. debug image I/O;
3. ONNX Runtime thread count;
4. execution provider selection;
5. preprocessing / tensor layout;
6. final live validation.

8 Warm-up Experiment

Three fresh ONNX sessions were tested. The first inference from each session was treated as cold and subsequent calls as warmed.

Metric	Result
Mean first/cold inference	454.620 ms
Mean warmed inference	340.214 ms
Reduction	25.17%

Warm-up was accepted as useful startup behaviour, but not used as a steady-state performance claim.

9 Debug Image I/O Experiment

The baseline callback saved a raw PNG, drew annotations, and saved an annotated PNG every frame. Two ROS parameters were added so these operations could be disabled without removing the debugging capability.

The control and full-debug-off runs produced:

Metric	Control A	Debug-off C
Mean callback latency	527.174 ms	369.902 ms
Mean processing FPS	1.963	2.906
Raw image saving	61.891 ms	0.000 ms
Annotation drawing	57.939 ms	0.000 ms
Annotated image saving	66.159 ms	0.000 ms

The debug-image path cost 185.988 ms/frame in the control. Removing it produced a 29.83% reduction in mean callback latency and a 48.06% increase in mean processing FPS.

This was the single largest practical optimisation found in Part B.

10 Threading Experiment

ONNX Runtime intra-operation threading was tested using the default configuration and explicit values of 1, 2, 4, and 8.

Thread setting	Mean inference
Default	74.378 ms
1	128.482 ms
2	107.521 ms
4	75.923 ms
8	168.730 ms

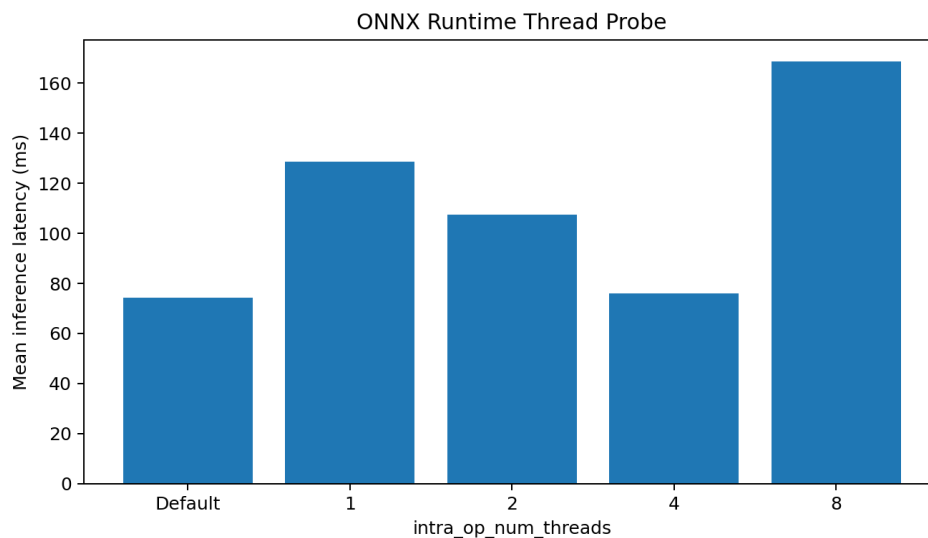


Figure 1: ONNX Runtime intra-operation thread probe.

The default setting was retained. Explicit thread counts did not improve the workload, and excessive threading significantly degraded performance.

11 Execution Provider Experiment

The available providers were CPUExecutionProvider and AzureExecutionProvider. An isolated probe first showed a lower mean inference latency with Azure + CPU fallback:

Provider	Mean inference	Median inference
CPU only	78.725 ms	78.478 ms
Azure + CPU fallback	70.596 ms	73.042 ms

A live controlled comparison then used debug image work disabled in both runs:

Metric	CPU only	Azure + CPU
Mean callback latency	369.902 ms	324.119 ms
Mean processing FPS	2.906	3.249
Mean ONNX inference	322.415 ms	285.869 ms

This produced 12.38% lower callback latency, 11.33% lower ONNX inference latency, and 11.80% higher processing FPS.

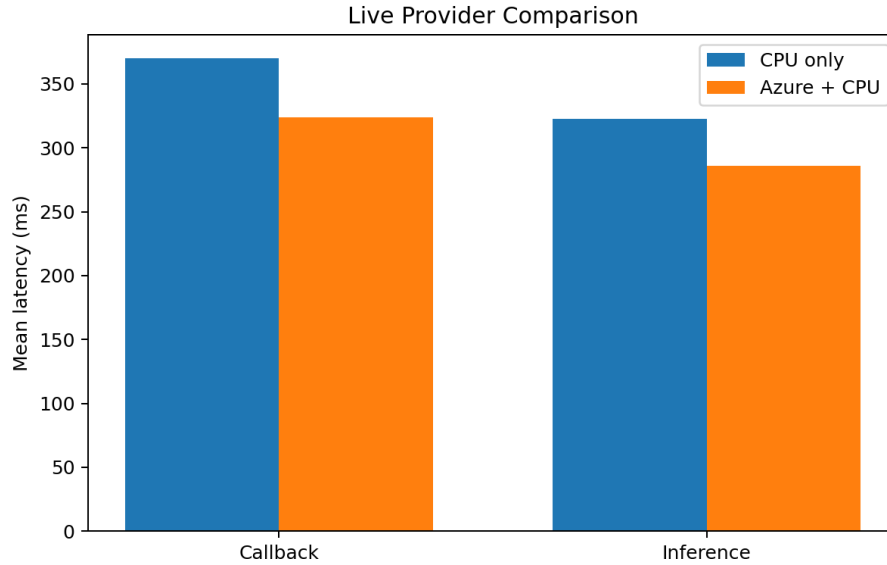


Figure 2: Live provider comparison. Azure + CPU fallback was selected.

The final provider mode was therefore set to AzureExecutionProvider with CPUExecution-Provider fallback.

12 Preprocessing and Tensor Layout Experiment

The original preprocessing produces a non-C-contiguous NCHW tensor after transpose. Two alternatives were investigated:

- explicitly copy the tensor into contiguous memory;
- use `cv2.dnn.blobFromImage`.

All tested tensors were numerically equivalent. However, the combined preprocessing + ONNX benchmark showed that the original path was still fastest:

Method	Mean combined latency
Original preprocessing	72.309 ms
Explicit contiguous copy	78.025 ms
<code>blobFromImage</code>	75.669 ms

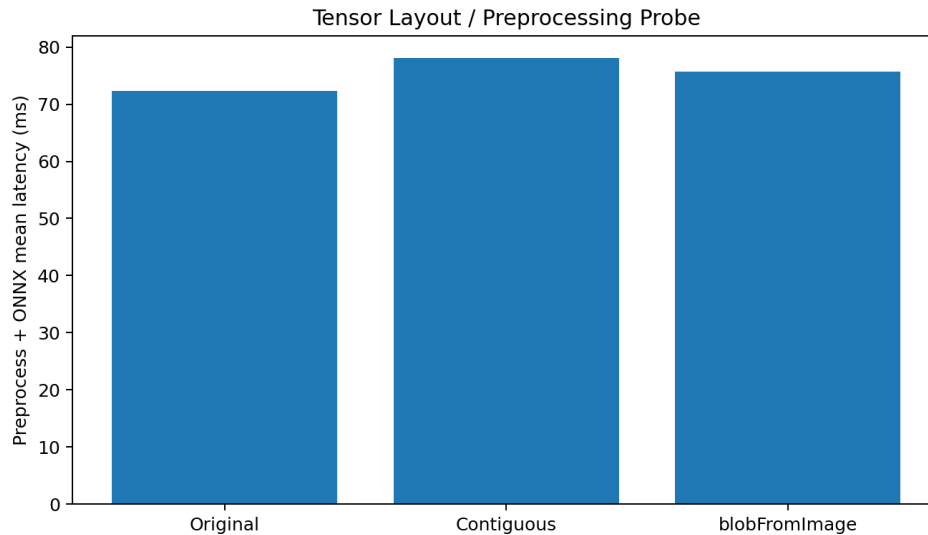


Figure 3: Tensor-layout alternatives. The original implementation remained fastest overall.

The original preprocessing was retained. This negative result is important because it prevented an apparently attractive low-level change from being adopted without end-to-end evidence.

13 Final Selected Configuration

The final implementation used:

- `save_raw_images = False;`
- `save_annotated_images = False;`
- `ort_provider_mode = azure_cpu;`
- default ONNX Runtime thread management;
- original preprocessing implementation;
- warm-up documented for startup behaviour.

The debug features remain configurable and can be re-enabled when visual inspection is required.

14 Final Controlled Validation

For the final Part B comparison, each run was standardised to 220 measured frames. The first 20 were excluded, leaving 200 steady-state frames in both the control and final runs.

Metric	Control A	Final E	Change
Mean callback latency	527.411 ms	382.364 ms	27.50% lower
Median callback latency	508.189 ms	346.844 ms	31.75% lower
P95 callback latency	679.593 ms	663.950 ms	2.30% lower
Mean processing FPS	1.962	2.887	47.13% higher

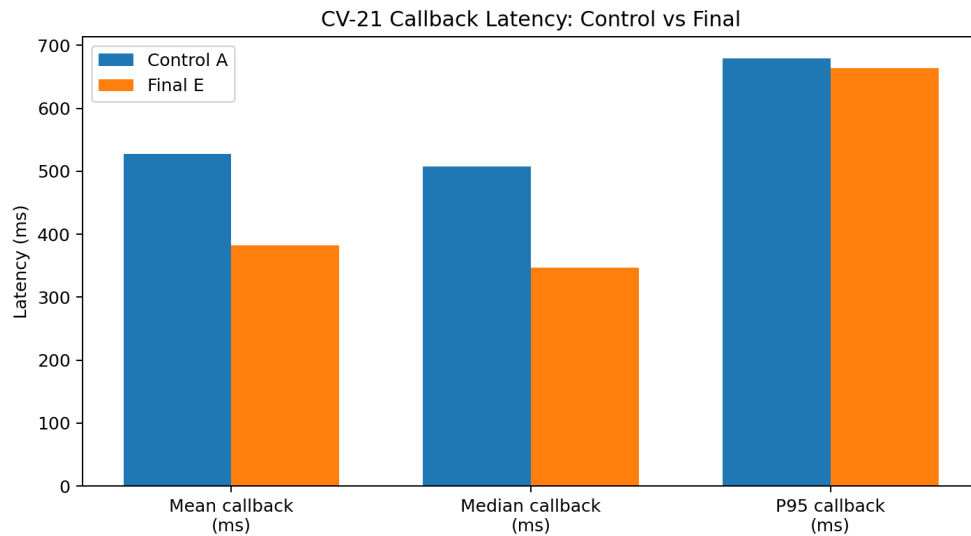


Figure 4: Final callback-latency comparison.

The final stage comparison was:

Stage	Control A	Final E
ROS image conversion	4.033 ms	3.702 ms
Raw PNG saving	61.969 ms	0.000 ms
Preprocessing	11.379 ms	11.328 ms
ONNX inference	293.871 ms	330.486 ms
Post-processing	18.345 ms	14.951 ms
Annotation drawing	57.821 ms	0.000 ms
Annotated PNG saving	66.200 ms	0.000 ms
Message construction	0.343 ms	0.274 ms
Metrics CSV writing	12.997 ms	20.460 ms
ROS publishing	0.334 ms	0.266 ms
Self-check	0.004 ms	0.003 ms

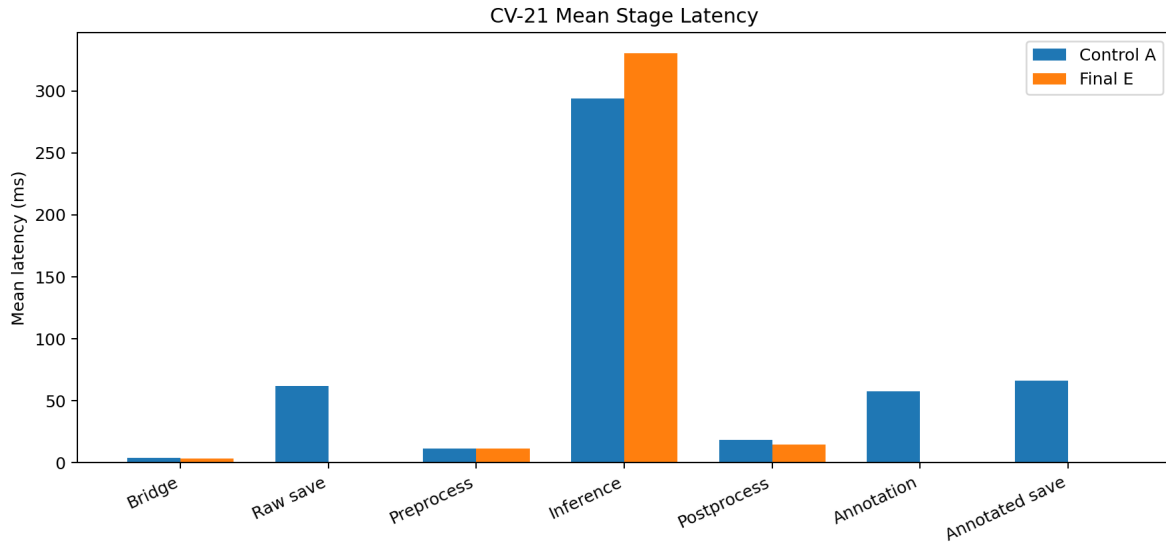


Figure 5: Mean stage latency in the final control and optimised runs.

The final run’s ONNX inference mean was higher than the control’s. This should not be interpreted as evidence against the provider optimisation because CPU/runtime latency varied substantially between live runs. Provider effectiveness is supported by the controlled C-to-D experiment, where provider mode was the intended changed factor and Azure + CPU reduced mean inference by 11.33%.

The final A-to-E comparison instead demonstrates the effect of the complete selected configuration. The largest end-to-end gain came from removing non-essential debug image work. The final run generated zero raw and zero annotated image files, confirming that the new defaults were active.

15 Relationship Between the Part A and Part B Baselines

Two sets of baseline numbers appear in the complete CV-21 evidence and they serve different purposes:

- The **archived Part A baseline** (545.371 ms mean callback, 1.943 FPS) is the formal original profiling dataset used to identify bottlenecks.
- The **Part B Run A control** (527.411 ms mean callback, 1.962 FPS in the final 200-frame comparison) was collected in the same optimisation campaign as the final Run E and is therefore used for the controlled before/after percentage calculation.

Keeping these roles separate avoids mixing measurements from different runtime sessions while preserving the original Part A evidence unchanged.

16 Engineering Decisions

The final CV-21 result is not simply a list of changes. It includes both accepted and rejected hypotheses:

Investigation	Decision	Reason
Warm-up	Retain as practice startup	Reduced first-inference mean by 25.17% in the probe.
Debug image I/O	Disable by default	Removed about 186 ms/frame of non-essential work in the control experiment.
Explicit ORT thread counts	Reject	Default thread management was fastest.
Azure + CPU provider	Accept	Improved both isolated and controlled live provider tests.
Explicit contiguous tensor	Reject	Combined preprocess + ONNX latency became slower.
<code>blobFromImage</code>	Reject	Numerically correct but slower than the original combined path.

This evidence-based selection process is the main technical value of Part B.

17 Evidence and Reproducibility

The work preserved:

- original baseline source copies;
- instrumented Part A source files;
- raw and steady-state benchmark CSVs;
- runtime environment information;
- warm-up, thread, provider, preprocessing, and layout probe logs;
- control and optimised live benchmark runs;
- final comparison text and plots;
- final optimised source snapshots and hashes.

The Part B evidence is organised under:

```
cv21_benchmark/part_b/
  configs/
  logs/
  plots/
```

```
results/  
source_snapshots/
```

18 Final Technical Outcome

CV-21 progressed from stage-level instrumentation and bottleneck identification to controlled optimisation experiments and final live validation.

The main final result from the controlled Part B comparison was:

- **27.50% lower mean callback latency;**
- **31.75% lower median callback latency;**
- **47.13% higher mean processing FPS;**
- zero per-frame raw or annotated debug image files under the final defaults.

The provider experiment separately demonstrated an 11.33% reduction in mean ONNX inference latency when moving from CPU-only execution to Azure + CPU fallback under the controlled live comparison.

The work also demonstrated that not every plausible optimisation should be adopted: explicit thread counts and alternative tensor-layout/preprocessing paths were rejected because the measurements did not support them.

19 Conclusion

CV-21 is complete as a single substantial technical contribution consisting of two linked phases. Part A created the instrumentation, established a reproducible baseline, and identified the bottlenecks. Part B converted those findings into controlled experiments, selected the useful changes, rejected ineffective alternatives, and validated the final configuration in the live ROS 2 / Gazebo pipeline.

The result is a more efficient perception pipeline together with a reproducible performance-engineering record that explains *why* the final configuration was chosen.